

not be orthonormal sets for an optimum solution to exist. Given an optimum pair (U_0, V_0) , one could change the basis of the column space of V_0 to a non-orthogonal one and adjust U_0 to the corresponding coordinates in the non-orthogonal basis system, so that the product $U_0 V_0^T$ does not change. To understand this point, we recommend the reader to solve the following problem:

Problem 8.3.1 *Let $D \approx Q_k \Sigma_k P_k^T$ be the rank- k SVD of D . The results in Chapter 7 show that $(U, V) = (Q_k \Sigma_k, P_k)$ represents an optimal pair of rank- k solution matrices to the unconstrained matrix factorization problem that is posed in this section. Show that $(U, V) = (Q_k \Sigma_k R_k^T, P_k R_k^{-1})$ is an alternative optimal solution to the unconstrained matrix factorization problem for any $k \times k$ invertible matrix R_k .*

8.3.1 Gradient Descent with Fully Specified Matrices

In this section, we will investigate a method that finds a solution to the unconstrained optimization problem with the use of gradient descent. This approach does not guarantee the orthogonal solutions provided by singular value decomposition; however, the formulation is equivalent and should (ideally) lead to a solution with the same objective function value. The approach also has the advantage that it can easily be adapted to more difficult settings such as the presence of missing values in the matrix. A natural application of this type of approach is that of matrix factorization in recommender systems. Recommender systems use the same optimization formulation as SVD; however, the resulting basis vectors of the factorization are not guaranteed to be orthogonal.

In order to perform gradient descent, we need to compute the derivative of the unconstrained optimization problem with respect to the parameters in the matrices $U = [u_{iq}]$ and $V = [v_{jq}]$. The simplest approach is to compute the derivative of the objective function J with respect to each parameter in the matrices U and V . First, the objective function is expressed in terms of the individual entries in the various matrices. Let the (i, j) th entry of the $n \times d$ matrix D be denoted by x_{ij} . Then, the objective function can be restated in terms of the entries of the matrices D , U , and V as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^d \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2$$

The quantity $e_{ij} = x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js}$ is the error of the factorization for the (i, j) th entry. Note that the objective function J minimizes the sum of squares of e_{ij} . One can compute the partial derivative of the objective function with respect to the parameters in the matrices U and V as follows:

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j=1}^d \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-v_{jq}) \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ &= \sum_{j=1}^d (e_{ij}) (-v_{jq}) \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i=1}^n \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-u_{iq}) \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \\ &= \sum_{i=1}^n (e_{ij}) (-u_{iq}) \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \end{aligned}$$

One can also express these derivatives in terms of matrices. Let $E = [e_{ij}]$ be the $n \times d$ matrix of errors. In the denominator layout of matrix calculus, the derivatives can be expressed as follows:

$$\begin{aligned}\frac{\partial J}{\partial U} &= -(D - UV^T)V = -EV \\ \frac{\partial J}{\partial V} &= -(D - UV^T)^T U = -E^T U\end{aligned}$$

The above matrix calculus identity can be verified by using the relatively tedious process of expanding the (i, q) th and (j, q) th entries of each of the above matrices on the right-hand side, and showing that they are equivalent to the (corresponding) scalar derivatives $\frac{\partial J}{\partial u_{iq}}$ and $\frac{\partial J}{\partial v_{jq}}$. An alternative approach that directly uses the matrix calculus identities of Chapter 4 is given in Figure 8.1. The reader may choose to skip over this derivation without loss of continuity.

The optimality conditions for this optimization problem are therefore obtained by setting these derivatives to 0. Therefore, we obtain the optimality conditions $DV = UV^T V$ and $D^T U = VU^T U$. These optimality conditions can be shown to hold for the solution obtained from SVD $U = Q_k \Sigma_k$ and $V = P_k$.

Problem 8.3.2 Let $Q_k \Sigma_k P_k^T$ be the rank- k truncated SVD of matrix D . Show that the solution $U = Q_k \Sigma_k$ and $V = P_k$ satisfies the optimality conditions $DV = UV^T V$ and $D^T U = VU^T U$.

A useful hint for solving the above problem is to use the spectral decomposition of SVD as a sum of rank-1 matrices.

Although the optimality condition leads to the standard SVD solution $D \approx [Q_k \Sigma_k] P_k^T$, one can also find an optimal solution by using gradient descent. The updates for gradient descent are as follows:

$$\begin{aligned}U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U + \alpha EV \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V + \alpha E^T U\end{aligned}$$

Here, $\alpha > 0$ is the learning rate.

The optimization model is identical to that of SVD. If the aforementioned gradient descent method is used (instead of the power iteration method of the previous chapter), one will typically obtain solutions that are equally good in terms of objective function value, but for which the columns of U (or V) are not mutually orthogonal. The power iteration methods yields solutions with orthogonal columns. Although the standardized SVD solution with orthonormal columns is typically not obtained by gradient descent, the k columns of U will span¹ the same subspace as the columns of Q_k , and the columns of V will span the same subspace as the columns of P_k .

The gradient-descent approach can be implemented efficiently when the matrix D is sparse by sampling entries from the matrix for making updates. This is essentially a *stochastic* gradient descent method. In other words, we sample an entry (i, j) and compute its error

¹This will occur under the assumption that the top- k eigenvalues of $D^T D$ are distinct. Tied eigenvalues result in a non-unique solution for SVD, which might sometimes result in some differences in the subspace corresponding to the smallest eigenvalue within the rank- k solution.

Consider the following objective function the $n \times d$ matrix D with rank- k matrices U and V :

$$J = \frac{1}{2} \|D - UV^T\|_F^2$$

Matrix calculus can be used for computing derivatives with respect to U and V after decomposing the Frobenius norm into row-wise vector norms or column-wise vector norms, depending on whether we wish to compute the derivative of J with respect to U or V . Let $\bar{X}_i$ be the i th row of D (row vector), $\bar{d}_j$ be the j th column of D (column vector), $\bar{u}_i$ be the i th row of U (row vector), and $\bar{v}_j$ be the j th row of V (row vector).

Then, the Frobenius norm can be decomposed in row-wise fashion as follows:

$$J = \frac{1}{2} \sum_{i=1}^n \|\bar{X}_i - \bar{u}_i V^T\|^2 = \frac{1}{2} \underbrace{\sum_{i=1}^n \bar{X}_i \bar{X}_i^T}_{\text{Constant}} - \sum_{i=1}^n \bar{X}_i V \bar{u}_i^T + \frac{1}{2} \sum_{i=1}^n \bar{u}_i V^T V \bar{u}_i^T$$

To compute the derivative of the two non-constant terms with respect to $\bar{u}_i$, we can use identities (i) and (ii) of Table 4.2(a) in Chapter 4. This yields the following:

$$\begin{aligned} \frac{\partial J}{\partial \bar{u}_i^T} &= -V^T \bar{X}_i^T + V^T V \bar{u}_i^T \\ \frac{\partial J}{\partial [\bar{u}_1^T \dots \bar{u}_n^T]} &= -V^T [\bar{X}_1^T \dots \bar{X}_n^T] + V^T V [\bar{u}_1^T \dots \bar{u}_n^T] \\ \frac{\partial J}{\partial U^T} &= -V^T D^T + V^T V U^T \\ \frac{\partial J}{\partial U} &= -DV + UV^T V = -(D - UV^T)V \end{aligned}$$

In order to compute the derivative with respect to V , one will need to decompose the squared Frobenius norm in J in column-wise fashion as follows:

$$J = \frac{1}{2} \sum_{j=1}^d \|\bar{d}_j - U \bar{v}_j^T\|^2 = \frac{1}{2} \underbrace{\sum_{j=1}^d \bar{d}_j^T \bar{d}_j}_{\text{Constant}} - \sum_{j=1}^d \bar{d}_j^T U \bar{v}_j^T + \frac{1}{2} \sum_{j=1}^d \bar{v}_j U^T U \bar{v}_j^T$$

One can again use identities (i) and (ii) of Table 4.2(a) to show the following:

$$\frac{\partial J}{\partial \bar{v}_j^T} = -U^T \bar{d}_j + U^T U \bar{v}_j^T$$

As in the previous case, one can put together the derivatives for different rows of V to obtain the following:

$$\frac{\partial J}{\partial V} = -D^T U + V U^T U = -(D - UV^T)^T U$$

Figure 8.1: Alternative derivation of factorization gradients using matrix calculus

e_{ij} . Subsequently, we make the following updates to the i th row $\bar{u}_i$ of U and the j th row $\bar{v}_j$ of V , which are also referred to as *latent factors*:

$$\begin{aligned}\bar{u}_i &\leftarrow \bar{u}_i + \alpha e_{ij} \bar{v}_j \\ \bar{v}_j &\leftarrow \bar{v}_j + \alpha e_{ij} \bar{u}_i\end{aligned}$$

One cycles through the sampled entries of the matrix (making the above updates) until convergence. The fact that we can sample entries of the matrix for updates means that we do not need fully specified matrices in order to learn the latent factors. This basic idea forms the foundations of recommender systems.

Problem 8.3.3 (Regularized Matrix Factorization) *Let D be an $n \times d$ matrix that we want to factorize using a rank- k decomposition into U and V . Suppose that we add the regularization terms $\frac{\lambda}{2}(\|U\|^2 + \|V\|^2)$ to the objective function $\frac{1}{2}\|D - UV^T\|_F^2$. Show that the gradient descent updates need to be modified as follows:*

$$\begin{aligned}U &\leftarrow U(1 - \alpha\lambda) + \alpha EV \\ V &\leftarrow V(1 - \alpha\lambda) + \alpha E^T U\end{aligned}$$

The entries of the matrices U and V can be initialized as follows. First, all $n \times k$ entries in U are independently sampled from a standard normal distribution, and then each column is normalized to the unit vector. Note that the matrix U contains roughly orthogonal columns, if n is large (see Exercise 18 of Chapter 1). The matrix V is selected to be $D^T U$. This approach ensures that UV^T yields $UU^T D$, where UU^T is (roughly) a projection matrix because of the approximate orthogonality of U . Thus, the initialized product is already closely related to the target matrix.

8.3.2 Application to Recommender Systems

Let D be an $n \times d$ ratings matrix representing the ratings of n users for d items. The (i, j) th entry in the matrix D is denoted by x_{ij} , and it represents the rating of user i for item j . The key distinguishing point of a recommendation application is that the vast majority of ratings are missing. This is because users do not specify the ratings of the vast majority of the items in collaborative filtering applications. An example of a ratings matrix with missing entries is shown in Figure 8.2. In other words, the value of x_{ij} is observed (known) for only a small subset of the entries. The goal of the recommendation problem is to predict the missing ratings from the known ones.

Let S represent the set of indices of all the observed ratings. Therefore, we have:

$$S = \{(i, j) : x_{ij} \text{ is observed}\} \quad (8.2)$$

As in the case of traditional matrix factorization, we would like to factorize the *incomplete* ratings matrix D with the use of only the entries in S . In the terminology of recommender systems, the $n \times k$ matrix U is referred to as the *user factor* matrix, and the $d \times k$ matrix V is referred to as the *item factor matrix*. Regularization is particularly important in the case of the collaborative filtering application because of the paucity of observed data. Therefore, an additional term $\frac{\lambda}{2}(\|U\|_F^2 + \|V\|_F^2)$ is added to the objective function.

Once the user and item factor matrices have been learned, the entire ratings matrix can be reconstructed as UV^T . In practice, we only need to reconstruct the (i, j) th entry of matrix D as follows:

$$\hat{x}_{ij} = \sum_{s=1}^k u_{is} \cdot v_{js} \quad (8.3)$$

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
TOM	1			5		2
JIM		5			4	
JOE	5	3		1		
ANN			3			4
JILL				3	5	
SUE	5		4			

Figure 8.2: A ratings matrix with missing ratings

Note the “hat” symbol (i.e., circumflex) on the rating on the left-hand side to indicate that it is a predicted value rather than an observed value. The error e_{ij} of the prediction is $e_{ij} = x_{ij} - \hat{x}_{ij}$ for ratings that are observed.

One can then formulate the objective function in terms of the *observed* entries in D as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{(i,j) \in S} \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^d \sum_{s=1}^k v_{js}^2$$

The main difference from the objective function in the previous section is the use of only observed entries in S for squared error computation, and the use of regularization. As in the previous section, we can compute the partial derivative of the objective function with respect to the various parameters as follows:

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) + \lambda u_{iq} \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i:(i,j) \in S} (e_{ij})(-u_{iq}) + \lambda v_{jq} \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \end{aligned}$$

One can also define these errors in matrix calculus notation. Let E be an $n \times d$ error matrix, which is defined to be e_{ij} for each observed entry $(i, j) \in S$ and 0 for each missing entry in the ratings matrix. Note that (unlike vanilla SVD), the error matrix E is already sparse because the vast majority of entries are not specified.

$$\begin{aligned} \frac{\partial J}{\partial U} &= -EV + \lambda U \\ \frac{\partial J}{\partial V} &= -E^T U + \lambda V \end{aligned}$$

Note that the form of the derivative is exactly identical to traditional SVD except for the regularization term and the difference in how the error matrix is defined (to account for missing ratings). Then, the gradient-descent updates for the matrices U and V are as follows:

$$\begin{aligned} U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U(1 - \alpha\lambda) + \alpha EV \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V(1 - \alpha\lambda) + \alpha E^T U \end{aligned}$$

Here, $\alpha > 0$ is the learning rate. The matrix E can be explicitly materialized as a sparse error matrix, and the above updates can be achieved using only sparse matrix multiplications. Although this approach is referred to as singular value decomposition in the literature on recommender systems (because of the relationship of unconstrained matrix factorization with the SVD optimization model), one will typically not obtain orthogonal columns of U and V with this approach.

8.3.2.1 Stochastic Gradient Descent

Stochastic gradient descent is introduced in Section 4.5.2 of Chapter 4. From the perspective of matrix factorization, the idea is to sample observed entries in S in order to perform the updates one at a time. Note that in the above updates, the (i, j) th error entry e_{ij} influences only the i th rows of U and V respectively. Stochastic gradient descent simply pulls out these updates on an entry-wise basis, rather than summing up the updates over all entries. Let $\bar{u}_i$ be a k -dimensional row vector containing the i th row of U and $\bar{v}_j$ be a k -dimensional row vector containing the j th row of V . Then, stochastic gradient descent cycles through each entry $(i, j) \in S$ in random order, and performs the following updates:

$$\begin{aligned} \bar{u}_i &\Leftarrow \bar{u}_i(1 - \alpha\lambda) + e_{ij}\bar{v}_j \\ \bar{v}_j &\Leftarrow \bar{v}_j(1 - \alpha\lambda) + e_{ij}\bar{u}_i \end{aligned}$$

Here, $\alpha > 0$ is the learning rate. Note that exactly $2k$ entries in the matrices U and V are updated for each observed entry in S . Therefore, a single cycle of stochastic gradient descent through all the observed ratings will make exactly $2k|S|$ updates. One starts by initializing the matrices U and V to uniform random values in $(0, M/\sqrt{k})$, where M is the maximum value of a rating. This type of initialization ensures that the initial product UV^T yields values in a similar order of magnitude as the original ratings matrix. One then performs the aforementioned updates to convergence. Stochastic gradient descent tends to converge faster than gradient descent, and is often the method of choice in recommender systems.

8.3.2.2 Coordinate Descent

The method of coordinate descent is introduced in Section 4.10 of Chapter 4. The basic idea in coordinate descent methods is to optimize one parameter at a time. In the case of matrix factorization, this amounts to optimizing a single parameter in U and V .

In the following, we will use the objective function J used in recommender systems (with incomplete matrices), because it is the more general form of the objective function. All notations such as the observed entry set S and the error e_{ij} are the same as those used in the previous section. By setting the partial derivative of the objective function J with respect to u_{iq} to 0, we obtain the following for each $i \in \{1 \dots n\}$ and $q \in \{1 \dots k\}$:

$$\begin{aligned}
\frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) + \lambda u_{iq} = 0 \\
u_{iq}(\lambda + \sum_{j:(i,j) \in S} v_{jq}^2) &= \sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq} \\
u_{iq} &= \frac{\sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq}}{\lambda + \sum_{j:(i,j) \in S} v_{jq}^2}
\end{aligned}$$

In the second step of the above algebraic manipulation, we added the quantity $\sum_{j:(i,j) \in S} u_{iq}v_{jq}^2$ to both sides in order to create a stable form of the update. The final form of the algebraic equation contains u_{iq} on both sides, and therefore it provides an iterative update. One can also derive a similar iterative update for each v_{jq} . The updates for the various values of u_{iq} and v_{jq} need to be performed sequentially as follows:

$$\begin{aligned}
u_{iq} &\leftarrow \frac{\sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq}}{\lambda + \sum_{j:(i,j) \in S} v_{jq}^2} \quad \forall i, q \\
v_{jq} &\leftarrow \frac{\sum_{i:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})u_{iq}}{\lambda + \sum_{i:(i,j) \in S} u_{iq}^2} \quad \forall j, q
\end{aligned}$$

One simply starts with random values of the parameters in the matrices U and V , and performs the above updates. One cycles through the $(m+n) \cdot k$ parameters in U and V with these updates until convergence is reached.

8.3.2.3 Block Coordinate Descent: Alternating Least Squares

Alternating least squares is a form of block coordinate descent, which is introduced in Section 4.10.2 of Chapter 4. This approach with an initial pair of matrices U and V , and then *alternates* between updating U and V , while keeping the other matrix fixed. Therefore, the process works as follows:

1. Keeping U fixed, we solve for each of the d rows of V by treating the problem as a least-squares regression problem. Only the observed ratings in S can be used for building the least-squares model in each case. Let $\bar{v}_j$ be the j th row of V . In order to determine the optimal vector $\bar{v}_j$, we wish to minimize $\sum_{i:(i,j) \in S} (x_{ij} - \sum_{s=1}^k u_{is}v_{js})^2$, which is a least-squares regression problem in $v_{j1} \dots v_{jk}$. The terms $u_{i1} \dots u_{ik}$ are treated as constant values, whereas $v_{j1} \dots v_{jk}$ are treated as optimization variables. Therefore, the k -dimensional vector $\bar{v}_j$ for the j th item are determined with least-squares regression. A total of d such least-squares problems need to be executed, and each least-squares problem has k variables. Because the least-squares problem for each item is independent, this step can be parallelized easily.
2. Keeping V fixed, solve for each of the n rows of U by treating the problem as a least-squares regression problem. Only the specified ratings in S can be used for building the least-squares model in each case. Let $\bar{u}_i$ be the i th row of U . In order to determine the optimal vector $\bar{u}_i$, we wish to minimize $\sum_{j:(i,j) \in S} (x_{ij} - \sum_{s=1}^k u_{is}v_{js})^2$, which is a least-squares regression problem in $u_{i1} \dots u_{ik}$. The terms $v_{j1} \dots v_{jk}$ are treated as constant values, whereas $u_{i1} \dots u_{ik}$ are treated as optimization variables. A total of n such least-squares problems need to be executed, and each least-squares problem has k variables. Because the least-squares problem for each user is independent, this step can be parallelized easily.